NON-STATIC MEMBER

Any member declared inside class block without prefixing ~~kitti~~ static modifier is known as Non-static member of a class.

In java, we have following Non-static members

1) Non-static variable
2) Non-static method
3) Non-static initializers
4) Non-static constructor.

NOTE :

1) The Non-static members get their memory allocated inside Object.
2) If we need to access any Non-static member, the first thing we must do is create an Object.

What is an Object?

An object is a block of memory created during the run-time inside heap area.

The object is also known as **INSTANCE OF A CLASS.**

The process of creating an object is known as **INSTANCIATION**

SYNTAX TO CREATE AN OBJECT:

```
new constructor of the class;
```

SYNTAX FOR CONSTRUCTOR OF THE CLASS:

```
ClassName()
```

Therefore, the actual syntax of Object creation is

```
new ClassName();
```

Program to find the default value of Non-static variable:

```
class N2
{
 int a;
 public static void main (String[] args)
   {
   System
   System.out.println (new N2().a);
   }
}
```

O/P:

0

▶ new :

* new is a keyword.
* It is an unary operator.
* It is used to create an object along with constructor of the class.
* new will create a block of memory inside heap area and it will return the address of it after the loading process is complet

* The type of address generated for the object will always be specific to its class. i.e. the address of object generated will always be in the below format:

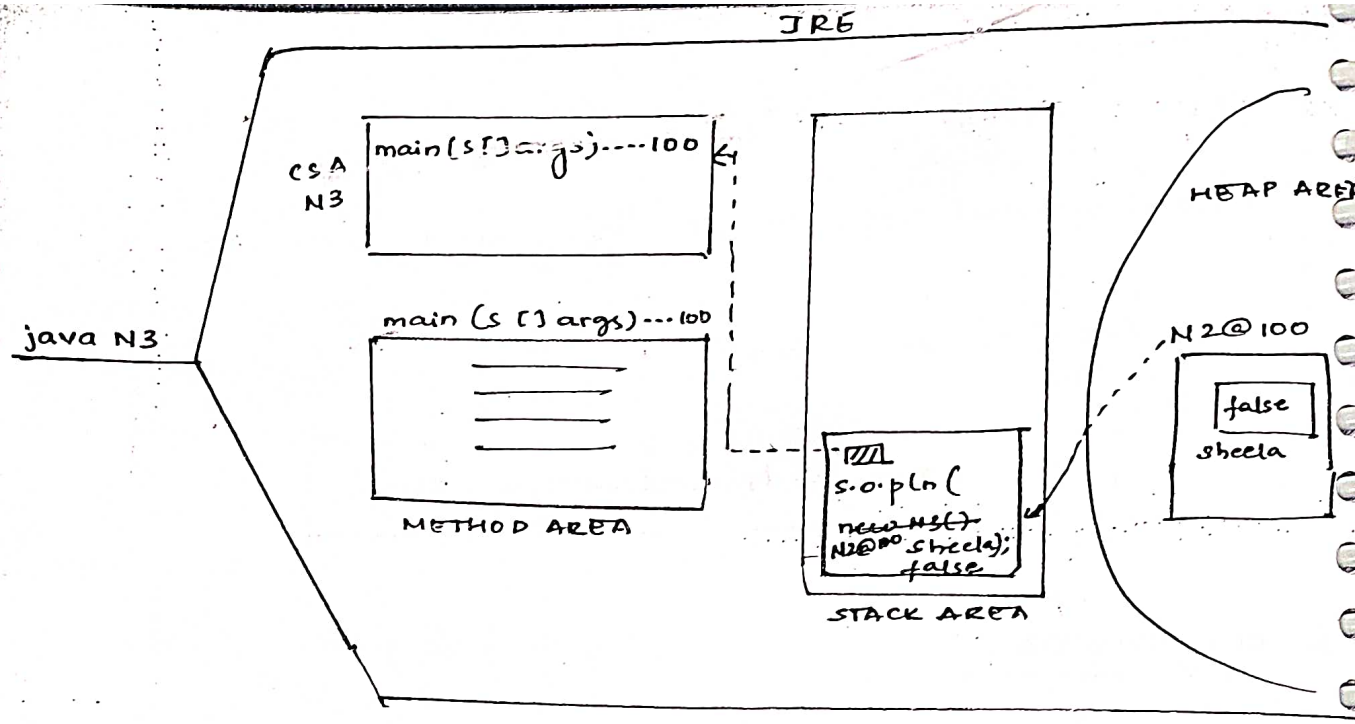
Classname @ hexadecimal value

CONSTRUCTOR:

Constructors are used to load the Non-static Members into the object.

Ex: class N3
{
boolean sheela;
public static void main (String[] args)
{
System.out.println (new N3().sheela);
}
}

o/p:
false



11/02/2022

NOTE:

1) We can have multiple objects created for one single class.

Ex: class N4
{
public static void main (String [] args)
{
System.out.println (new N4()); // N4@15db9742
System.out.println (new N4()); // N4@6d06d69c
}
}

2) Every time we create a new object, it will create a new block of memory in different addresses.

3) Every time we create an object, the ~~static~~ non-static members will be loaded into an object.

NON-PRIMITIVE DATA TYPE:

* They are user defined data types which are used to store the following things:

1) The default values of Non-primitive data type i.e, null

2) Address of an Object.

* The variables which are created using non-primitive data types are known as **REFERENCE VARIABLE.**

REFERENCE VARIABLE:

A variable which is created for Non-primitive data type ~~to store~~ is known as Reference variables.

The Reference variables are used to store address of Objects.

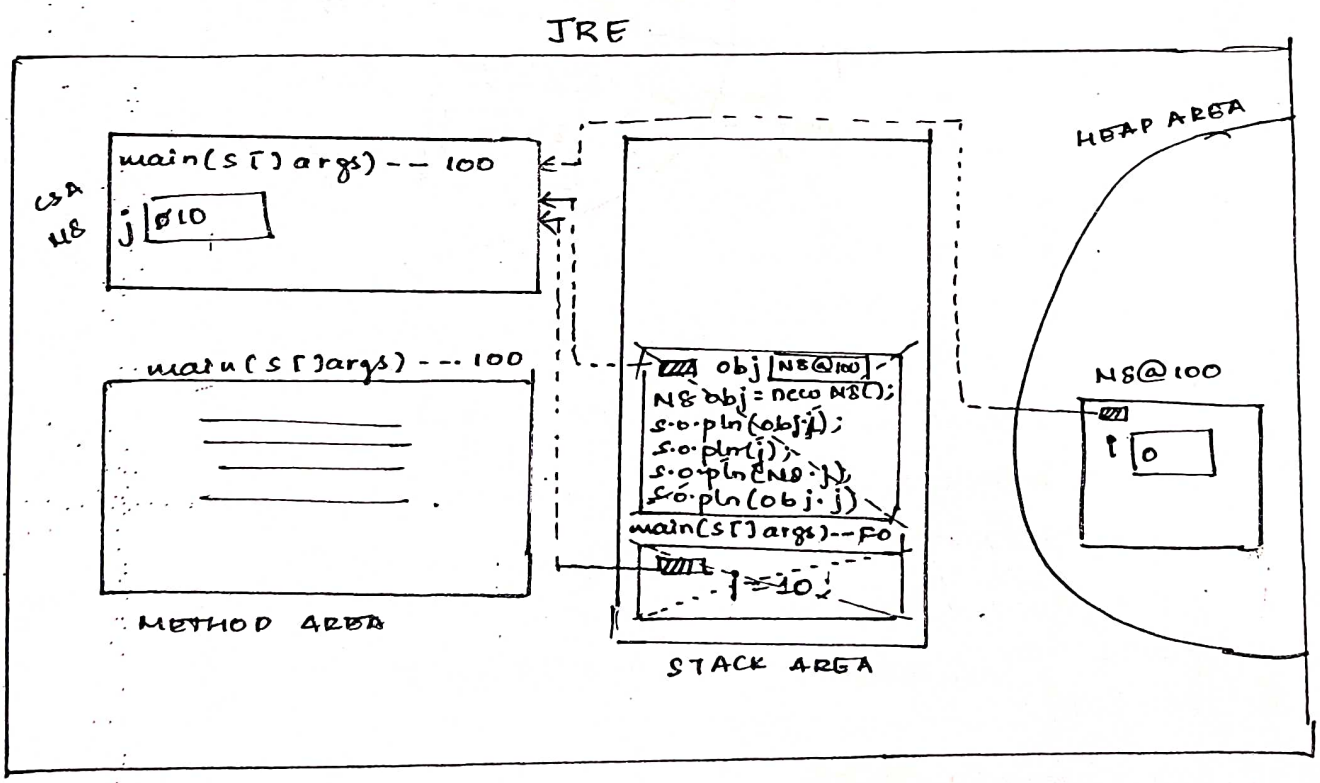
SYNTAX TO CREATE A REFERENCE VARIABLE:

```
ClassName  identifier;
```

NOTE: In java we ~~have~~ following can make use of the following members to create non-primitive data types:

Ex:

```
class N8
{
static int j=10;
int i;
public static void main(String[] args)
{
N8 obj = new N8();
System.out.println(obj.i);
System.out.println(j);
System.out.println(N8.j);
System.out.println(obj.j);
}
}
```



o/p:

0
10
10
10

Ex. for creation of Reference variable:

```
class P2
{
  static P2 obj1; // static Reference variable of type P2
  P2 obj2; // Nonstatic Reference variable of type P2
  {
  public static void main (String [] args)
  {
    P2 obj3 ; // local reference variable of type P2
    obj3 = new P2();
  }
}
```

NOTE:

Inside static context, we can access static members in 3 ways:

1) Directly with the help of member name.

2) With the help of class name & access operator.

3) With the help of address of the object.

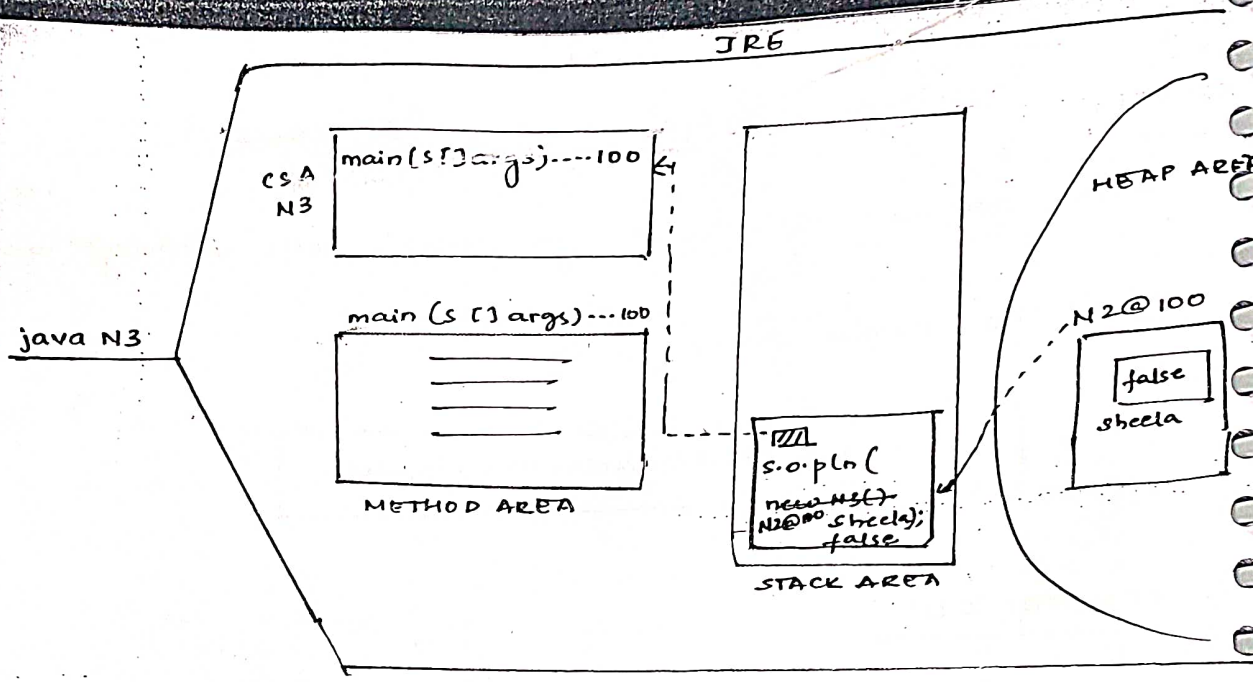
Inside a static context, we can access Non-static members in following ways:

1) Only with the help of address of the object.

Ex

c

{



11/02/2022

NOTE:

1) We can have multiple objects created for one single class.

Ex: class N4
{
public static void main (String [] args)
{
System.out.println (new N4()); // N4@15db9742
System.out.println (new N4()); // N4@6d06d69c
}
}

2) Every time we create a new object, it will create a new block of memory in different addresses.

3) Every time we create an object, the ~~static~~ non-static members will be loaded into an object.

NON-PRIMITIVE DATA TYPE:

* They are user defined data types which are used to store the following things:

1) The default values of Non-primitive data type i.e. null

2) Address of an Object

* The variables which are created using non-primitive data types are known as **REFERENCE VARIABLE.**

REFERENCE VARIABLE:

A variable which is created for Non-primitive data type ~~to store~~ is known as Reference variables.

The Reference variables are used to store address of Objects.

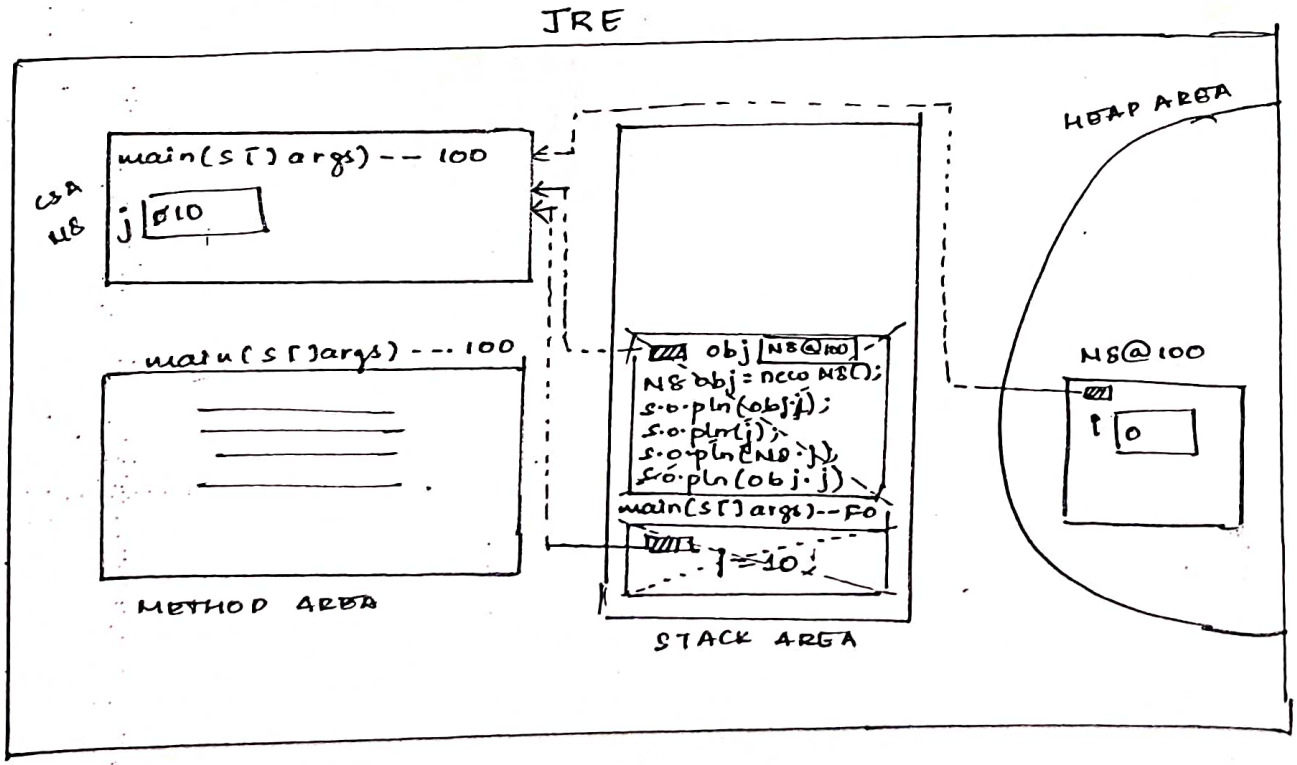
SYNTAX TO CREATE A REFERENCE VARIABLE:-

```
ClassName  identifier;
```

NOTE: In java we ~~have~~ following can make use of the following members to create non-primitive data types:

Ex:

```
class N8
{
static int j=10;
int i;
public static void main(String[] args)
{
N8 obj = new N8();
System.out.println(obj.i);
System.out.println(j);
System.out.println(N8.j);
System.out.println(obj.j);
}
}
```



O/P:

0
10
10
10

* for creation of Reference variable:

```
class P2
{
  static P2 obj1; // static Reference variable of type P2
  P2 obj2; // Nonstatic Reference variable of type P2
  {
  public static void main (String [] args)
  {
    P2 obj3 ; // local reference variable of type P2
    obj3 = new P2();
  }
}
```

NOTE:

Inside static context, we can access static members in 3 ways:

1) Directly with the help of member name.

2) With the help of class name & access operator.

3) With the help of address of the object.

Inside a static context, we can access Non-static members in following ways:

1) Only with the help of address of the object.

Ex

c

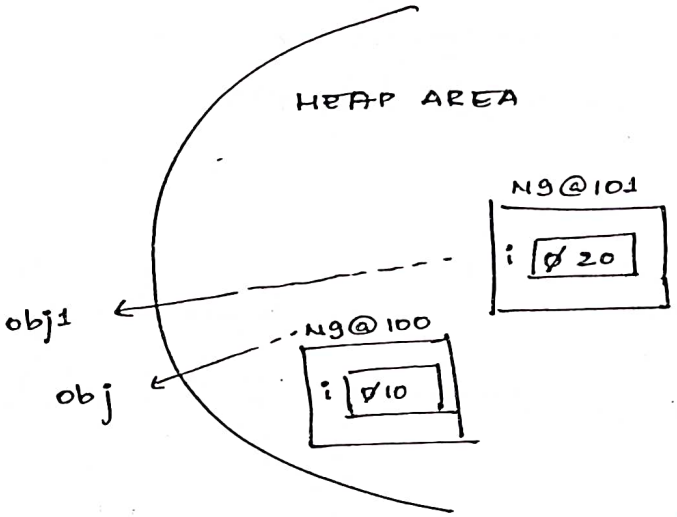
{

Ex 2:

```
class N9
{
int i;
public static void main (String [] args)
{
 N9 obj = new N9();
 obj.i = 10;
  N9 obj1 = new N9();
 obj1.i = 20;
 System.out.println (obj.i);
 System.out.println (obj1.i);
 }
}
```

Otp:

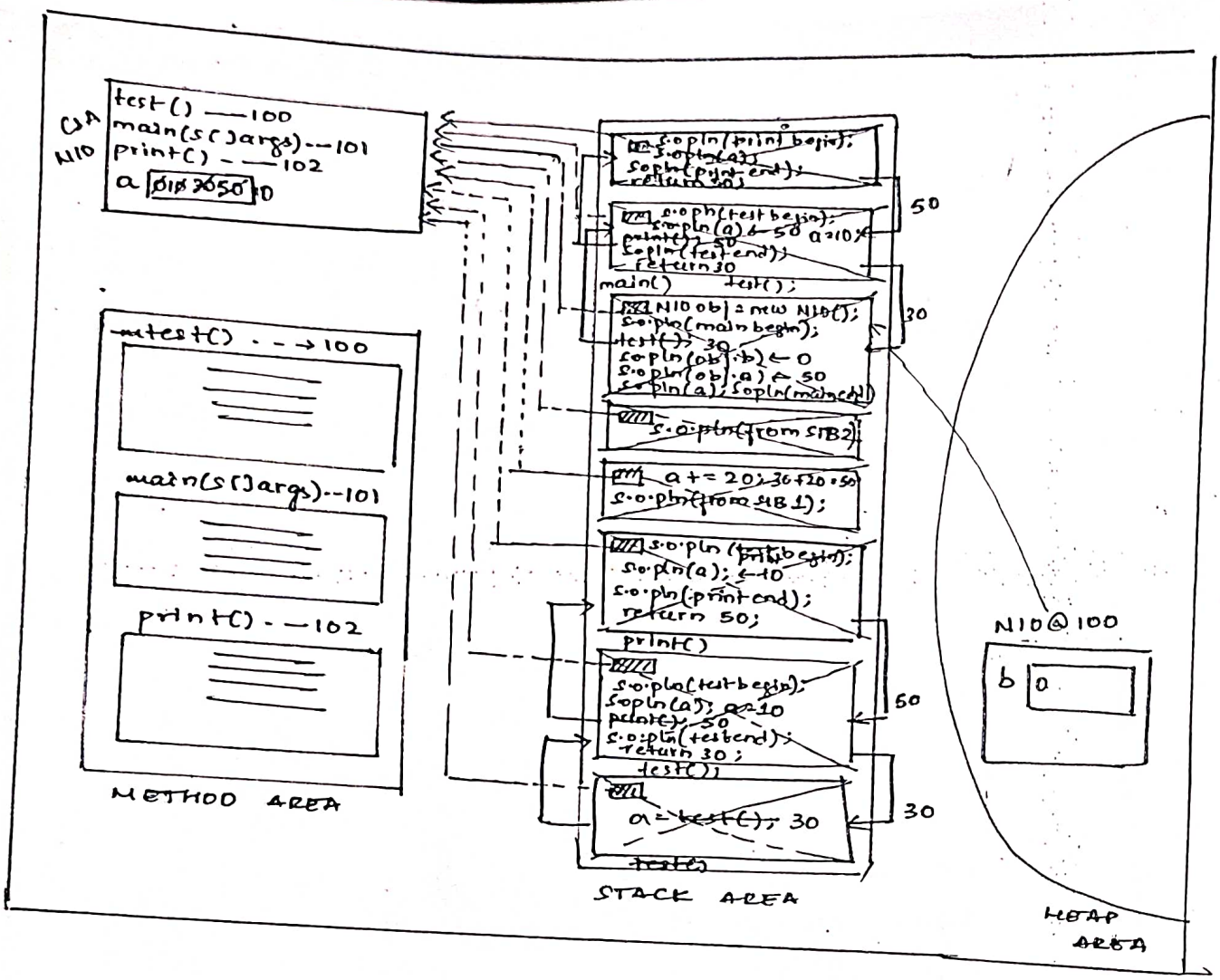
10
20



Ex

Ex: class N10

```
{
static int a = test();
int b;
public static int test();
{
System.out.println("test begin");
System.out.println(a);
a=10;
print();
System.out.println("test end");
return 30;
}
static
{
a+=20;
System.out.println("from SIB-1");
}
public static void main(String[] args)
{
N10 obj = new N10();
System.out.println("main begin");
test();
System.out.println(obj.b);
System.out.println(obj.a);
System.out.println(a);
System.out.println("main end");
}
public static int print()
{
System.out.println("test begin");
System.out.println(a);
System.out.println("test end");
return 50;
}
static
{
s.o.p.ln("from SIB-2");
}}
```



O/P:

Test begin

0

print

test begin

10

print end

Test end

from SIB-1

from SIB-2.

Main begin

Test begin

50

print begin

50 10

print end

Test end

0

$0

$0

Main end

12/02/2022

NOTE:-

i) Whenever we use new operator an object will be created so we can have multiple object created for some class.

ii) We will have non-static members loaded into each and every object created.

iii) So, we have same number of copies of non-static members created for no of objects created.

[Eg: 10 copies → 10 objects].

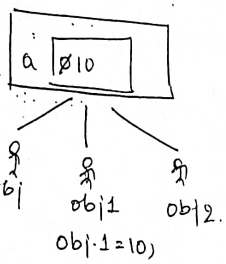
DIFFERENCE BETWEEN STATIC VARIABLES & NON-STATIC VARIABLE:-

STATIC VARIABLE	NON-STATIC VARIABLE
1) Static variables are prefixed with static modifier	Non-static variables are not prefixed with static modifier.
2) They get their memory allocated inside class static area.	They get their memory allocated inside object created in the heap area.
3) We can access in 3 ways inside static context:	Non static variable can be accessed inside static context:

a) Directly with the help of variable name

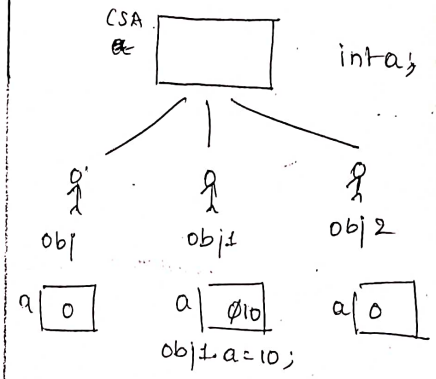
b) With the help of class name.

c) With the help of object address.

(iv) 

If we want stagnant value, we use static

c) with the help of object address only.



If we want different values in the container use non-static.

Eg:
```
class N12
{
static int a;
int b;
public static void main (String[] args)
{
N12 obj = new N12();
N12 obj1 = new N12();
N12 obj2 = new N12();
obj.a = 10;
obj.b = 20;
System.out.println (obj.a);
```

```
System.out.println(obj1.a);
System.out.println(obj2.a);
System.out.println(obj.b);
System.out.println(obj1.b);
System.out.println(obj2.b);
}
}
```

NON- STATIC METHOD

A method which is not prefixed with static modifier is known as Non-static method.

* Non- static methods get their memory allocated inside the object of the class.
* A block which belongs to non-static method is called as non-static context.
* Non-static methods can be accessed inside static context of same class using only one way ie object address.

SYNTAX :

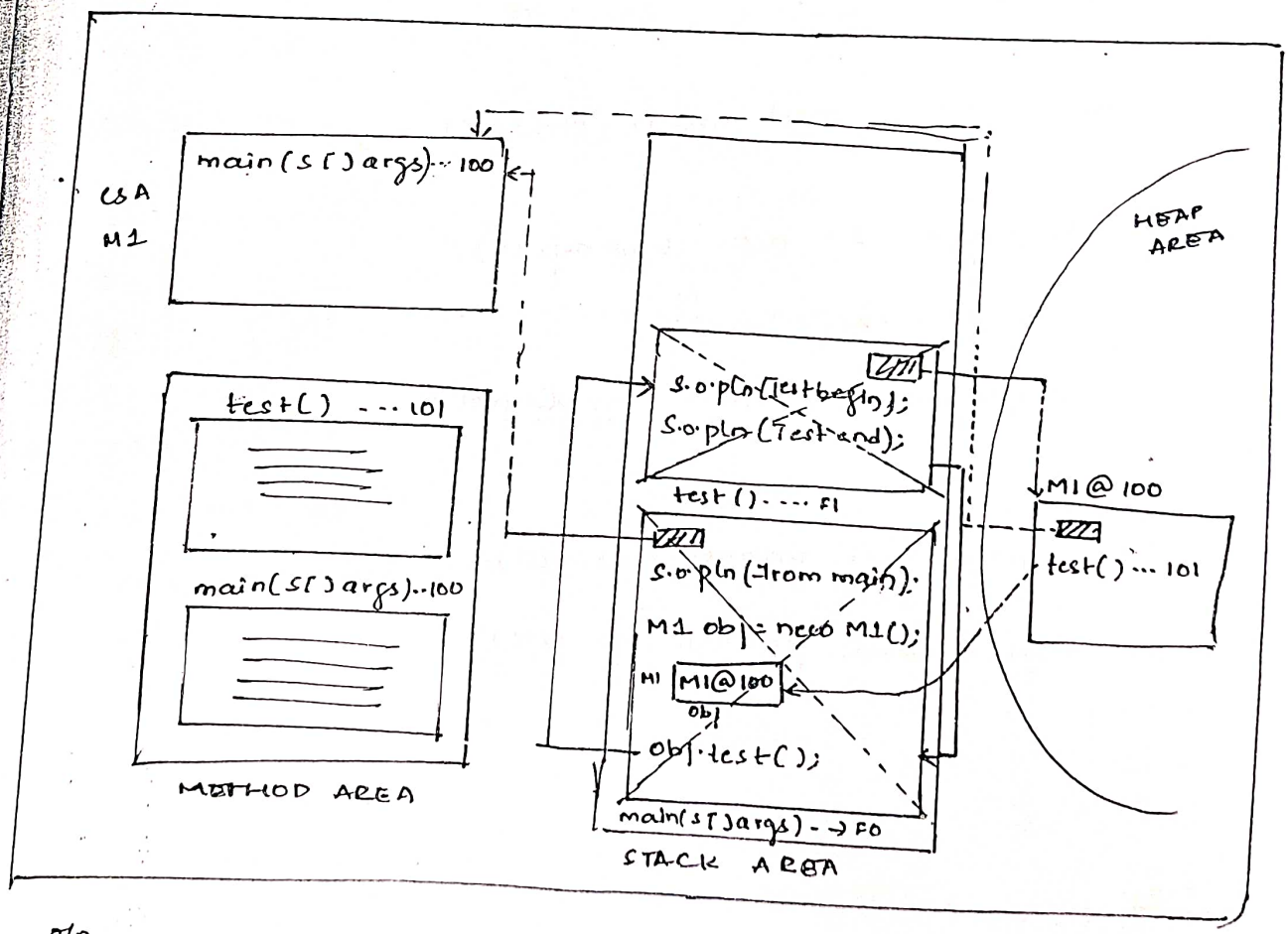
```
Eg: public void main (String [] args)
```

Eg:- cl:

```
MethodName ( )
{
//statements;
}
```

Eg:

```
class M1
{
public void test()
{
    System.out.println("Test begin");
    System.out.println("Test end");
}
public static void main(String[] args)
{
    System.out.println("From main");
    M1 obj = new M1();
    obj.test();
}
}
```



Op:

From main

Test begin

Test end

15/02/2022

NOTE :

In java, the states of the object are represented with the help of Non-static variables and the behaviours will be represented with the help of Non-static methods.

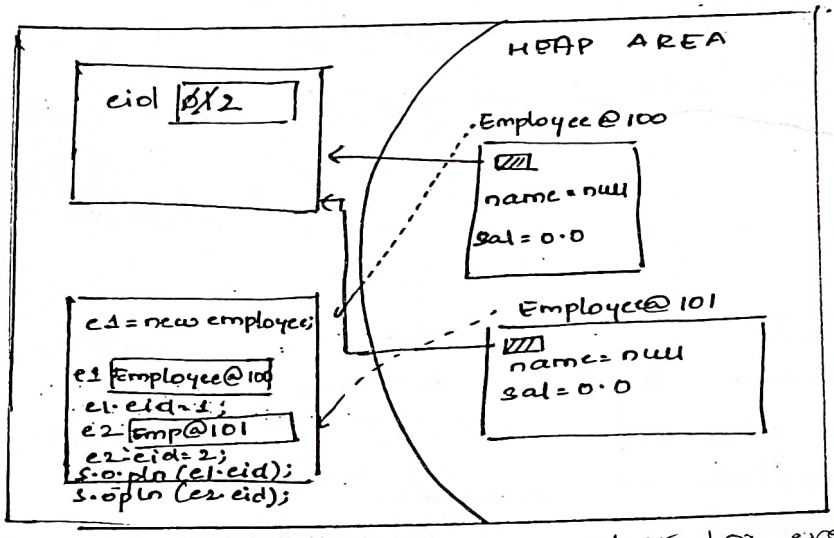
Ex

```
class Employee
{
static int eid;
String name;
double sal;
}
class Driver1
{
public static void main (String[] args)
  {
    Employee e1 = new Employee();
    e1.eid = 1;
    Employee e2 = new Employee();
    e2.eid = 2;
    System.out.println (e1.eid);
    System.out.println (e2.eid);
  }
}
```

Op :

1
1

TRACING :



In the above Ex since eid has be made static w can have only o copy of eid ~~that~~ throughout the application.

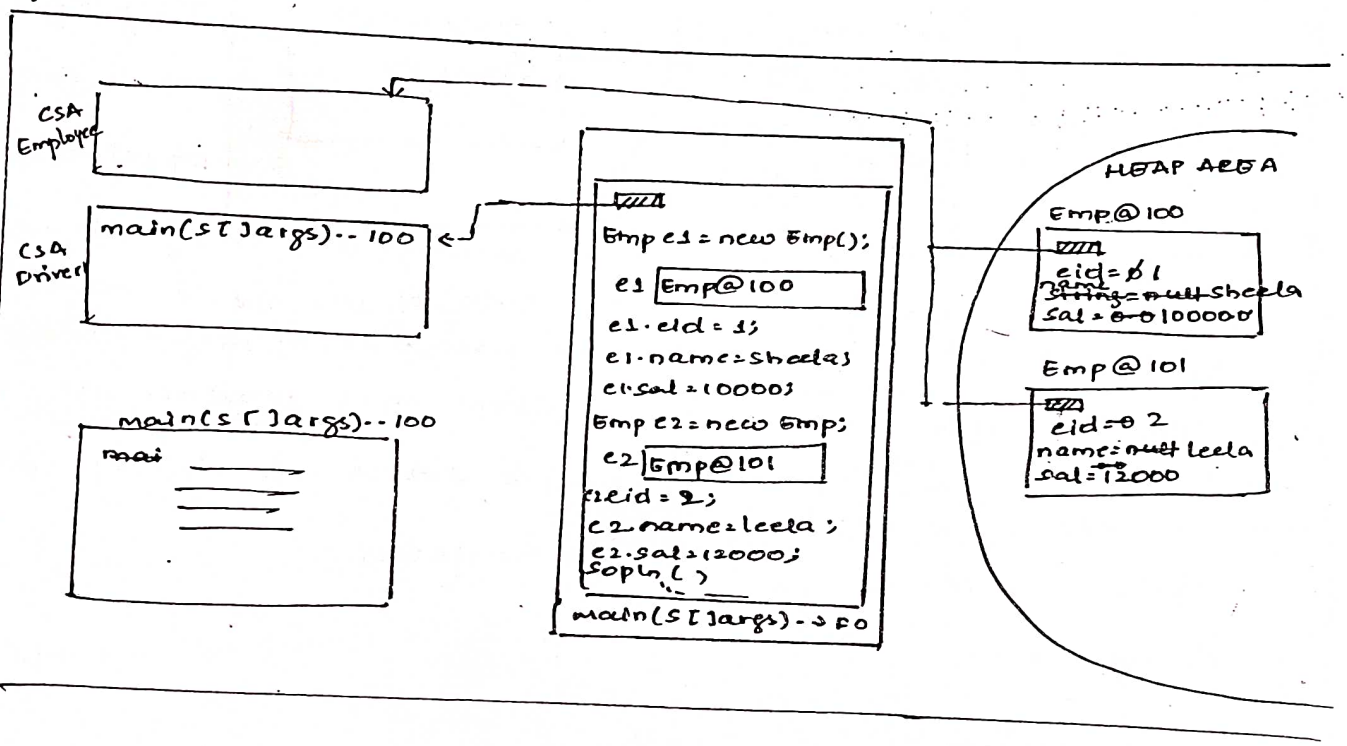
Therefore it will be a common member for every object created of Employee class. ie The eid's of all the employee will b updated to the latest eid assigned in the application.

To overcome this problem we make all the states of objects as Non static variables.

Ex:

```
class Employee
{
 int eid;
 String name;
 double sal;
}
class Driver1
{
public static void main(String[] args)
{
  Employee e1 = new Employee();
  e1.eid = 1;
  e1.ename = "Sheela";
  e1.sal = 10000;
  Employee e2 = new Employee();
  e2.eid = 2;
  e2.ename = "Leela";
  e2.sal = 12000;
  System.out.println(e1.eid);
  System.out.println(e1.name);
```

```
System.out.println(e1.sal);
System.out.println(e2.eid);
System.out.println(e2.ename);
System.out.println(e2.sal);
}
}
```



o/p:

1

sheela

10000.0

2

Leela

12000.0

NON-STATIC CONTEXT:

Any block or Any instruction which belongs to Non-static member of a class is known as Non-sta context

Eg: 1. The block of a Non-static method.

2. Non-static declare & initialization statement

NOTE:

1. We can use the Non-static members of the clas: Inside Non-static context directly

2. We can use static members of the class inside Non-static context directly.

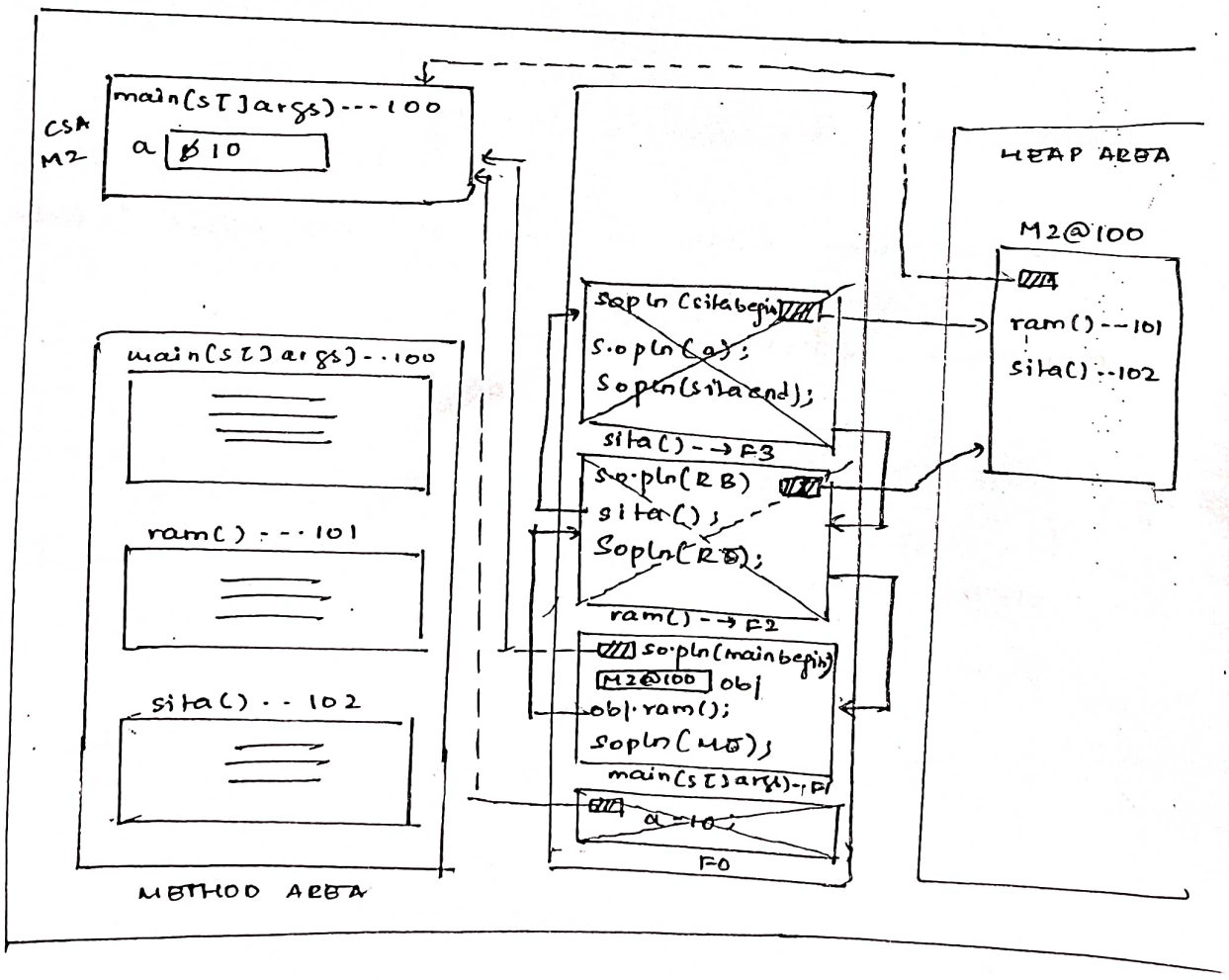
Eg:
```
class M2
{
 static int a=10;
 public void ram()
 {
  System.out.println("Ram begin");
  sital();
  System.out.println("Ram end");
 }
```

```
public static void main (String [] args)
{
System.out.println ("Main begin");
M1 obj = new M1();
obj.ram();
System.out.println ("Main end");
}

public void sita()
{
System.out.println ("sita begin");
System.out.println (a);
System.out.println ("sita end");
}
}
```

o/p :

Main begin
Ram begin
Sita begin
10
Sita end
Ram end
Main end



Eg:

```
class Student
{
String name;
int sid;
double percentage;
public void display()
{
System.out.println("Student name : "+ name);
System.out.println("Student id : "+ sid);
System.out.println("Student percentage : "+ percentage);
}System.out.println("==========");
}
class Driver3
{
public static void main(String[] args)
{
Student s1 = new Student();
s1.name = "Dinga";
s1.sid = 1;
s1.percentage = 40;
Student s2 = new Student();
s2.name = "Dingi";
s2.sid = 2;
s2.percentage = 80;
s1.display();
s2.display();
}
}
```

O/P:

Student name: Dinga

Student id : 1

Student percentage: 40.0

==========

Student name: Dingi

Student id: 2

Student percentage: 80.0

Q. Design a class inside which we must have a static method, static variable, both static initializers, a non-static variable & a non-static method inside which both static & non-static members must be used.

```
class P1 Design
{
 ~~static var =~~
 Static int var = 10;
 static
 {
 System.out.println("from SIB-1");
 }
 int var2;
 public void test()
 {
 System.out.println("from test begin");
 System.out.println(var) → var2 = 50;
 System.out.println(var2);
 System.out.println("test end");
 }
```

```
public static void main (String [] args)
{
System.out.println ("Main begin");
Design obj = new Design();
obj.test();
System.out.println ("Main end");
}
}
```